Submitted By:

Masood ul Hassan – 2025-CYS-52

Assignment:

PacMan Project

Submitted To:

Sir Shahmeer Nawaz

For the fulfillment of,

Object-Oriented Programming Lab

Department of Computer Engineering

University of Engineering and Technology, Lahore

Introduction:

I created a console based PacMan Game. The player moves using Keyboard inputs and eats food while the ghosts mioving randomly.

GitHub Link:

https://github.com/Masood251/PacMan

Classes:

	Class	Purpose
1	Position	Stores x and y coordinates
2	Food	Represents food pellets
3	Obstacle	Represents walls
4	PacMan	Controls movement,lives,score
5	Ghost	Controls enemy movement
6	Maze	Store the game board and food
7	GameBoard	Run the main loop and handles inputs
8	PacmanGame	Starts the game

Implementation:

I implemented this PacMan game using Java programming language. The project contains 8 classes that work together to run the game.

Development Environment:

Language: Java
IDE: IntelliJ IDEA
JDK Version: Java 17

How I Implemented Each Class:

1. Position Class:

This class stores x and y coordinates. I used it to keep track of where PacMan, Ghosts, and Obstacles are located on the maze.

2. Food Class:

This class represents the dots that PacMan collects. Each food object has a position and an eaten flag. When PacMan moves over a food dot, the eat() method marks it as eaten.

3. Obstacle Class:

This class represents walls in the maze. PacMan and Ghosts cannot move into obstacle positions.

4. PacMan Class:

This is the player character. I implemented methods for movement, score tracking, and lives management. The tryMove() method checks if a move is valid before updating position.

5. Ghost Class:

This class controls the enemy ghosts. I used the Random class to generate random directions. Each ghost tries up to 20 random moves to find a valid direction.

6. Maze Class:

This class loads the game board from a string array. It stores the grid layout, food array, and obstacle array. The eatFoodAt() method removes food when PacMan collects it.

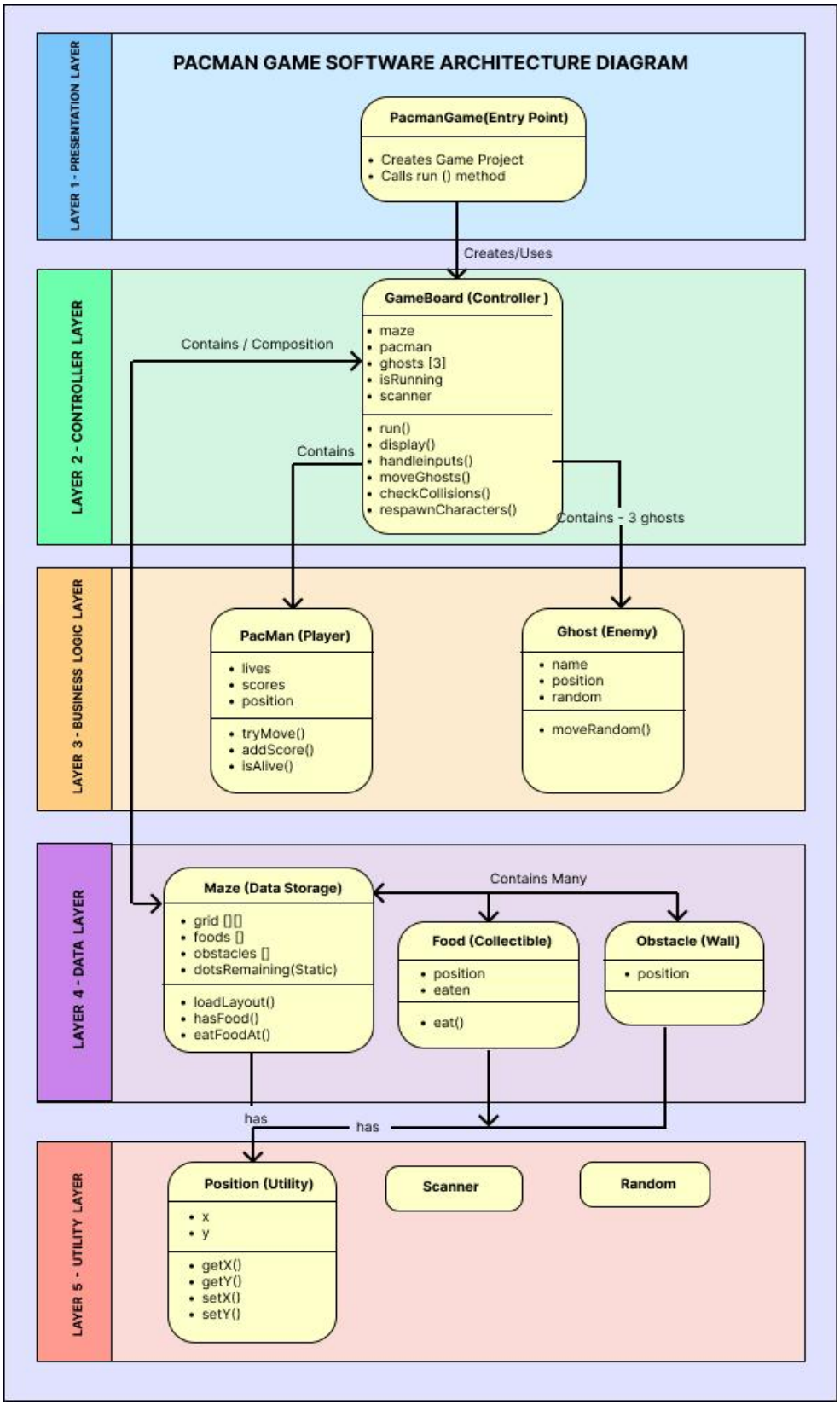
7. GameBoard Class:

This class runs the main game loop. It handles user input using Scanner, renders the game screen, moves ghosts, and checks collisions between PacMan and ghosts.

8. Main Class:

This class contains only the main() method which creates a Game object and starts the game.

Architecture Diagram:



How The Game Works:

Step 1: The game starts with PacMan at position (1,1) with 3 lives and 0 score.

Step 2: The player presses keys to move:

- ^ key moves up
- v key moves down
- < key moves left
- > key moves right

Step 3: PacMan moves around the maze collecting dots. Each dot gives 1 point.

Step 4: Ghosts move randomly around the maze.

Step 5: If a ghost touches PacMan, he loses one life and respawns.

Step 6: The game ends when:

- Player collects all dots (Win)
- Player loses all 3 lives (Game Over)

Classes Diagram:

1. Position Class:

Variables:

- x : int
- y : int

Methods:

- Position (int x, int y)
- getX ()
- getY ()
- setX (int x)
- setY (int y)

2. Food Class:

Variables:

- p : Position
- eaten : boolean

Methods:

- Food(int x, int y)
- getX ()
- getY ()
- isEaten ()
- Eat ()

3. Obstacle Class:

Variables:

- p : Position

Methods:

- Obstacle(int x, int y)
- getX ()
- getY ()

4. PacMan Class:

Variables:

- p : Position
- lives : int
- score : int

Methods:

- PacMan(int lives, int x, int y)
- tryMove (int direction, char [][] mazeGrid, Obstacle [] obstacles)
- isValidMove(int x, int y, char [][] mazeGrid, Obstacle [] obstacles }
- addScore ()
- isAlive ()
- respawn (int startX, int startY)
- getX ()
- getY ()
- getLives ()
- setLives (int lives)
- getScore ()

5. Ghost Class:

Variables:

- name : String
- p : Position
- random : Random

Methods:

- Ghost (String name, int x, int y)
- moveRandom (char [][] mazeGrid, Obstacle [] obstacles)
- isValidMove(int x, int y, char [][] mazeGrid, Obstacle [] obstacles }
- getName ()
- getX ()
- getY ()
- setPosition (int x, int y)

6. Maze Class:

Variables:

- dotsRemaining : int
- grid : char [][]
- height : int
- width : int
- foods : Food []
- obstacles : Obstacle []

Methods:

- Maze (int height, int width)
- loadLayout(String [] layout)
- hasFood (int x, int y)

- eatFoodAt (int x, int y)

- getDotsRemaining ()
- getGrid ()
- getHeight ()
- getWidth ()
- getObstacles ()

7. GameBoard Class:

Variables:

- maze : Maze
- pacman : PacMan
- ghosts : Ghost []
- isRunning : boolean
- sc : Scanner

Methods:

- Game ()
- initializeGame ()
- dispaly ()
- handleInput ()
- moveGhosts ()
- checkCollisions ()
- respawnCharacters ()
- run ()

8. PacmanGame Class:

Variables:

No Variable

Method:

- Main method

Testing Results:

I tested all game features to ensure everything works correctly. The results are shown below:

Test 1: Press ^ key - Expected Result: PacMan moves up - Actual Result: Working

Test 2: Press v key - Expected Result: PacMan moves down - Actual Result: Working

Test 3: Press < key - Expected Result: PacMan moves left - Actual Result: Working

Test 4: Press > key - Expected Result: PacMan moves right - Actual Result: Working

Test 5: Move into a wall - Expected Result: PacMan stops and does not go through - Actual Result: Working

Test 6: Collect a food dot - Expected Result: Score increases by 1 point - Actual Result: Working

Test 7: Ghost touches PacMan - Expected Result: Lives decrease by 1 - Actual Result: Working

Test 8: PacMan loses all 3 lives - Expected Result: Game Over message appears - Actual Result: Working

Test 9: PacMan collects all dots - Expected Result: You Win message appears - Actual Result: Working

Conclusion:

I made a PacMan game in Java that works in the console. The player can move PacMan using keys, collect food dots, and avoid ghosts. Each dot gives 1 point. If a ghost catches PacMan, he loses one life. The game ends when the player either collects all dots and wins or loses all 3 lives.

Through this project, I learned how to use classes, objects, arrays, loops, collision detection, and random movement for ghosts. All features of the game are working properly.